Palm OS Programming Recipes

Data Manager: Basic Database Operations

Palm OS applications can create, delete, open, and close databases just as applications on a desktop operating system can create, delete, open, and close files. Using the Palm OS Data Manager is similar to using standard file I/O functions in a desktop application, except that the database data is broken down into multiple records instead of being stored in one contiguous chunk as it is in a file. Palm OS database data is stored in multiple, finite-sized records that can be scattered throughout the handheld's memory space. This means that adding, deleting, or resizing database records doesn't require you to move other records around in memory. **Opening and Closing the Database** Before you can read or write database records, you must open the database. In the process of opening the database, most applications create one if the database doesn't already exist. Applications should close their databases whenever the user switches to a new application. The `AppStop()` function is a good place to close databases opened in your application.

To open and close a database, do the following:

1. Declare a global variable of type `DmOpenRef` to hold the database reference (a *handle*). Applications generally use a global variable to hold the database handle, making the handle accessible throughout the application.
2. In function `AppStart()`, call `DmOpenDatabaseByTypeCreator()` to try to open the database. If `NULL` is returned from `DmOpenDatabaseByTypeCreator()`, indicating that the database does not exist, call `DmCreateDatabase()` to create the database and then open it using `DmOpenDatabaseByTypeCreator()`. Your code should look something like the following (note that the `FrmAlert()` calls are for testing purposes and would likely not appear in production code):

```
// Tell the user that the DB is being opened
FrmAlert (OpenDBAlert);
gDatabase = DmOpenDatabaseByTypeCreator( kType, kCreator,
        dmModeReadWrite);
if (!gDatabase) {
        // Tell the user that we're creating a new DB
    FrmAlert (DBCreatedAlert);
    err = DmCreateDatabase(kCardNumber, kName, kCreator,
        kType, false);

    if (!err) {
        gDatabase = DmOpenDatabaseByTypeCreator(kType, kCreator,
                dmModeReadWrite);
        if (!gDatabase)
            err = DmGetLastErr();
    }
}
```

3. Call `DmCloseDatabase()` from within `AppStop()` to close the database. Be sure that the database is indeed open before you attempt to close it, as is done in the following code:

```
FrmAlert (CloseDBAlert); // Tell the user that we're closing the DB

if (gDatabase)
    DmCloseDatabase(gDatabase);
```

Tip: When debugging database code, you can use a tool such as `RsrcEdit` to explore the newly created database. **Creating and Deleting Records** To create new database records, you use `DmNewRecord()` and then either `DmWrite()`, `DmSet()`, or `DmAttach()` to add the new record to the database. `DmAttach()` is useful for moving records between databases as well as creating new records.

To remove records from a database, you use `DmRemoveRecord()`, `DmArchiveRecord()`, or `DmDeleteRecord()`. `DmRemoveRecord()` deletes record information from the database headers along with the record data itself. `DmArchiveRecord()` and `DmDeleteRecord()` are necessary if the handheld database is to be synchronized with a desktop database.

To create and remove a record from an open database, do the following:

1. Using `DmNewRecord()` to create records and `DmWrite()` to write newly-created records to the database, write a function similar to the following:

```
static void CreateRecord()
{
    MemHandle h;
    UInt16 index = 0;
    DBRecordType r;  // this is our database record

    h = DmNewRecord(gDatabase, &index, sizeof(r));

    // fill the record with data
    StrCopy(r.FirstName, "Jun");
    StrCopy(r.LastName, "Zhou");
    StrCopy(r.PhoneNum, "1 555 545 4434");

    if (h) { // could fail due to insufficient memory!
        MemPtr p = MemHandleLock(h);
        Err err = DmWrite(p, 0, &r, sizeof(r));

        MemPtrUnlock(p);
        DmReleaseRecord(gDatabase, index, true);
    }
}
```

2. Write a function that removes records from the database using `DmRemoveRecord()`. It should look something like this:

```
static void RemoveRecord()
{
    if (DmNumRecords(gDatabase) > 0) {
        Err err = DmRemoveRecord(gDatabase, 0);
    }
```

```
    }
```

Tip: Databases designed for desktop applications often use records containing fixed-length strings. If the strings in your application tend to vary in length, reserving a fixed number of bytes can result in wasted space that, in a handheld application, you may not be able to afford. You can save space by packing strings. This is usually done by concatenating the record's strings before writing the record and unpacking them when reading the record's contents. For sample code that uses this technique, see the published source code of the Palm PIM applications. **Reading and Updating Database Records** To read or update the contents of a database you use `DmQueryRecord()`, or `DmGetRecord()` and `DmReleaseRecord()`. For read-only access to database records, use `DmQueryRecord()`. If you need to edit database records, call `DmGetRecord()`, and when you are done with the record call `DmReleaseRecord()`. Unlike `DmQueryRecord()`, `DmGetRecord()` marks the record *busy*. `DmReleaseRecord()` then clears the busy bit, updates the modification number of the database, and marks the record *dirty* if the `dirty` parameter is `true`. Because `DmQueryRecord()` doesn't set the record's busy bit, it isn't necessary to call `DmReleaseRecord()` when you are done reading the record contents.

The following code shows how to obtain a record's contents with `DmQueryRecord()`:

```
    MemHandle h;
    UInt16 index = 0;

    // get the handle to the record and set busy bit
    h = DmQueryRecord(gDatabase, index);
    if (h) { // could fail due to insufficient memory!
        DBRecordType *p = (DBRecordType *)MemHandleLock(h);

        // do something with the record's contents here

        MemPtrUnlock(p);
    }
```

When editing a record, use `DmWrite()` and `DmSet()` to alter the record's contents. You can update an entire record or only a portion of a record. For instance, the following code shows how to update only one field within a structured record:

```
static void EditRecord()
{
    MemHandle h;
    UInt16 index = 0;

    // get the handle to the record and set busy bit
    h = DmGetRecord(gDatabase, index);
    if (h) { // could fail due to insufficient memory!
        DBRecordType *p = (DBRecordType *) MemHandleLock(h);

        // write only the changed fields of record
        Err err = DmWrite(p, OffsetOf(DBRecordType, LastName),
                          "Knopp", 20);
        MemPtrUnlock(p);
        DmReleaseRecord(gDatabase, index, true);
    }
}
```

Wrap-Up For more information on the functions discussed in this document, see the *Palm OS API Programmer's Reference*. >For a sample application that uses the code presented in this document, click on one of the following links. The "Code Added" comments in the sample application source code identify the code discussed above.

- Windows
- Mac